

Glen Olinger - GEOG485 - 25 July 2026

Columbia River Bar Channel Sea Floor Analysis

Objective

The objective of this project is to create a Python-based tool that automates seafloor change analysis. The analysis is intended to understand how the seafloor is changing from both erosion and deposition perspectives.

It is intended that the tool can be run automatically at some periodicity and execute the workflow without user interaction.

Area of Interest

This tool targets the Columbia River Bar channel, which is maintained by the U.S. Army Corps of Engineers for deep-draft vessel transit between the Pacific Ocean and the Columbia River.

Data source: U.S. Army Corps of Engineers (USACE) Hydrographic Surveys:

<https://www.arcgis.com/apps/dashboards/4b8f2ba307684cf597617bf1b6d2f85d#>

Note: The modules and execution scripts are designed to be flexible enough to adapt to other areas of interest and related analyses.

Approach

1. Investigate the data.
2. Develop the workflow:
 1. Download survey data.
 2. Preprocess survey data.
 3. Perform erosion and deposition analysis.
 4. Generate a report.
3. Develop standalone Python tools for each workflow task.
4. Convert workflow task tools into importable modules.
5. Develop a main execution tool that runs all tasks.
6. Develop a batch script for automating the running the workflow.

Detailed Workflow

1. Download new surveys.
 1. Define a time window of interest.
 2. Determine whether surveys in that window have already been downloaded.
 3. Download new surveys to a specified directory.
2. Preprocess the new surveys.
 1. Unzip survey data so it can be used by the analysis step.
 2. Maintain processed data in a specified directory.
3. Analyze the surveys.
 1. Define the comparison time window (number of months).
 2. Compute deposition and erosion between two survey rasters.

3. Maintain analysis results in a specified directory.
4. Generate final reports.
 1. Create a map layout of analysis results.
 2. Export the map layout to PDF.
 3. Maintain reports in a specified directory.
5. Execution.
 1. Create an execution script that:
 - Defines required parameter inputs.
 - Runs workflow steps 1 to 4.
 2. Create a Windows batch script that runs the Python execution script for use with Windows Task Scheduler.

Pseudo Code

1. Download new surveys.

Function Def: check_and_download_surveys(bbox, outdir, survey_filter, limit, start_date, end_date):

```

Check inputs
Create output directory if needed
Load list of already-downloaded survey IDs

Initialize counters and paging values

Loop while still under download limit:
  Query survey service for next page
  If no results:
    break

  Loop for each survey in results:
    Extract survey metadata (id, name, url, date)
    Skip invalid or duplicate records
    Apply text/date filters
    Skip already-downloaded surveys

    If survey is eligible:
      Try download ZIP to output directory
      Update counters/status

  Move to next page

Print summary
Return True if any new surveys were downloaded, else False

```

2. Preprocess the new surveys.

Function Def: is_zip_already_extracted(zip_path, output_dir): Try to open ZIP Collect all file entries (ignore directories) If ZIP is invalid or has no file entries: Return False

```
For each file entry:
    If corresponding file does not exist in output_dir:
        Return False

Return True
```

Function Def: preprocess_surveys(input_dir, output_dir): If input_dir does not exist: Print message Return False

```
Create output_dir if needed
Get all ZIP files from input_dir

If no ZIP files found:
    Print message
    Return False

Initialize counters (extracted, skipped)

For each ZIP file:
    If already extracted:
        Mark skipped
        Continue

    Try to extract ZIP to output_dir
    If successful:
        Increment extracted
    Else:
        Increment skipped

Print processing summary
Return True if any ZIP was extracted, else False
```

3. Analyze the surveys.

Function Def: analyze_surveys(months_diff, source_dir, output_dir, cell_size, threshold):

```
Ensure output directory exists
Validate source directory exists

Find all dated survey geodatabases in source_dir
Validate there are enough surveys to compare

Select latest survey
Compute target date = latest survey date minus months_diff
Select comparison survey closest to target date (from older surveys)

Build paths to required raster inputs
Validate required rasters exist
```

Create run-specific output folder and set ArcPy workspace

Project both rasters to a common coordinate system

Align raster grids and cell size

Compute overlap extent

Compute change raster (latest minus past)

Save raw change raster

Apply threshold filter to remove small/noisy changes

Save filtered change raster

Derive deposition and erosion rasters from filtered change

Save outputs

Print completion status

4. Generate final reports.

Function def: export_single_report(project_path, layout_name, raster_path, pdf_path, source_dir_name, zoom_out_factor, resolution):

Validate project file exists

Validate raster file exists

Open ArcGIS Pro project

Locate requested layout

Update "Data" text element with source_dir_name

Select map frame (prefer "Extent" frame if available)

Remove old raster layers from map

Add current filtered raster layer

Set camera extent to raster

Optionally zoom out by zoom_out_factor

Save project changes

Export layout to PDF

Return success/failure status

Function def: gen_report(analyzed_dir, reports_dir, project_path, layout_name, zoom_out_factor, resolution):

Ensure reports directory exists

Validate analyzed_dir exists

List analysis run folders inside analyzed_dir

If none found, stop

Initialize summary counters

```
For loop each run folder:
    Build expected raster path and report PDF path

    If report already exists:
        Skip
        Continue

    If raster missing:
        Skip
        Continue

    Call export_single_report(...)
    Update success/failure counters

Print summary
Return overall status
```

5. Execution.

- Import workflow modules.
- Define needed input parameters.
- Call module workflow functions.
 1. **Download:** *check_and_download_surveys(bbox, outdir, survey_filter, limit, start_date, end_date).*
 2. **Preprocess:** *preprocess_surveys(input_dir, output_dir).*
 3. **Analyze:** *analyze_surveys(months_diff, source_dir, output_dir, cell_size, threshold).*
 4. **Reports:** *gen_report(analyzed_dir, reports_dir, project_path, layout_name, zoom_out_factor, resolution).*